

Phoenix Merchandise Application documentation

Release 1.0

720090306

Mar 10, 2024

1	README	1
1.1	Phoenix Merchandise Application Description	1
1.2	Installation and Setup Instructions	1
1.3	Usage Instructions	2
1.4	Software Design	7
1.5	Maintaining Software	9
1.6	References	10
2	Application Code	11
2.1	main module	11
2.2	constants module	11
2.3	views package	11
2.4	controllers package	12
2.5	Models package	13
	Python Module Index	19
	Index	21

README

1.1 Phoenix Merchandise Application Description

The Phoenix Merchandise Application's purpose is to be used by Phoenix, the Concert Merchandising Concession company, to provide the sales function for touring bands at certain concert venues around the country.

The application provides 3 main functions:

1. Loading in Concert Merchandise for Phoenix to sell
2. Creating Purchases for Merchandise sold to customers at concerts
3. Viewing a breakdown of sales for Merchandise sold

This application can be used by Phoenix Managers and Sales Staff to first load in Merchandise stock at the start of the day and then sell this Merchandise throughout the day, then finally at the end of the day reviewing a breakdown of the stock sold.

The application allows the user to do the following tasks:

- View, Add, Update and Delete Merchandise stored in the Phoenix Database
- View, Create and Cancel Purchases made for merchandise
- View a breakdown of the amount sold for each piece of merchandise and the total to pay the touring band and the commission made by Phoenix

1.2 Installation and Setup Instructions

To run the Phoenix Merchandise Application on your local computer, please follow the steps below:

(These steps assume that you have received a zipped copy of the project)

Installation:

- Ensure Visual Studio Code is installed on your computer
- Ensure Python 3.10+ is installed on your computer
- Ensure you have a Python Plugin installed in Visual Studio Code

Setup:

1. Unzip the project and save in an easily accessible location your computer
2. Open the unzipped project into Visual Studio Code

3. Create a virtual environment and activate it by following these commands:

- In your terminal run:

```
python -m venv .env
```

- The cd into .env then cd into Scripts and run the following command to activate your virtual environment:

```
.\activate
```

4. Once your virtual environment is activated, you can run the command to install the requirements:

```
pip install -r requirements.txt
```

Running the Application:

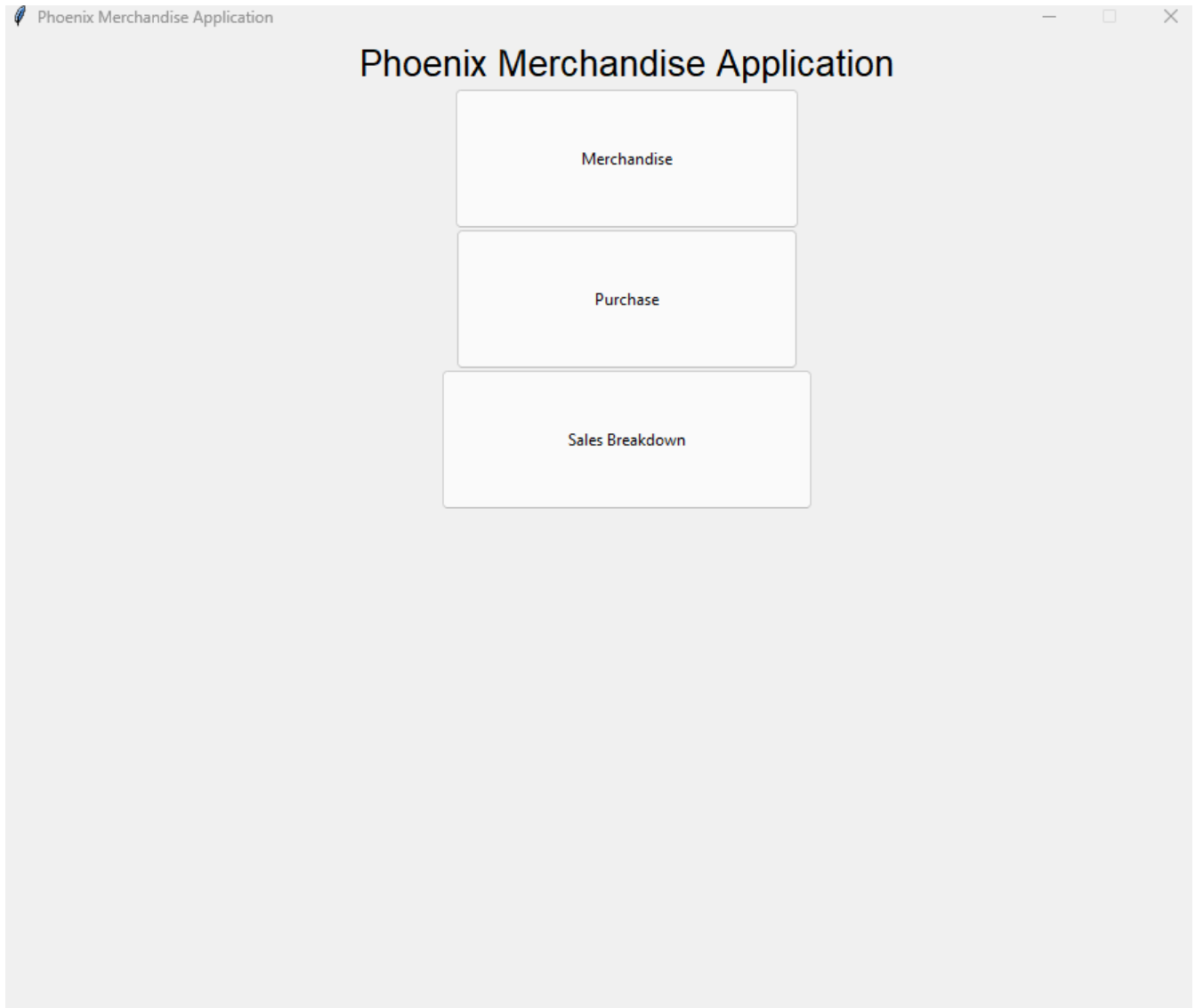
1. From the Project Directory (phoenix-app), cd into mycode
2. Then once you're in the mycode directory and you're in your virtual environment, run the command:

```
python main.py
```

1.3 Usage Instructions

Below are a set of instructions which will guide all users (Managers, Sales Staff and Software Developers) on how to use the application.

When you first run the application, you will see the Home Page.



From this page you can click which page you would like to visit using the buttons.

1.3.1 1) Merchandise Page

When you click the Merchandise button, you are taken to the Merchandise page. This is where managers can load in merchandise at the start of the day and Sales Staff can view the merchandise they have in stock.

ID	Product Type	Description	Price	Quantity
1	T-Shirt	A black T-shirt	5.99	30
2	Hoodie	A red Hoodie	10.99	50
3	Cap	A blue Cap	3.99	40

Record

Price Product Type Description Quantity

ID

Commands

[Back to Home Page](#)

The table shows all the existing records in Phoenix database for merchandise that has been loaded in. Below are some instructions to complete common actions:

Select a record:

1. Click the record in the table you want to Select (it will turn blue once selected)
2. Click the "Select Record" button in the Commands section

Delete a record:

1. Follow the steps above to select the record you want to remove
2. Press the "Remove Record" button in the Commands section and you will see it be removed from the table

Update a record:

1. Follow the steps above the select the record you want to update
2. Edit the fields after they have been populated in the boxes below the table
3. Don't edit the ID field

4. Once you have changed the fields, press the “Update Record” button

Add a record:

1. Fill the entry boxes below the table with the item you want to add
2. Do not fill in the ID field as that it automatically generated when added to database
3. Once you have populated the boxes, press the “Add Record” button to add the record

1.3.2 2) Purchase Page

If you click the Purchase Button from the Home Page, you will be taken to the Purchase Page.

The Purchase Page is where Sales staff can create Purchases. You can also view past purchases made and cancel purchases.

Phoenix Merchandise Application

Purchases

List of Purchases Made

ID	Product Type	Description	Price	Quantity	Total Cost	Date
1	T-Shirt	A black T-shirt	5.99	5	29.95	2024-02-27

Cancel Purchase

Purchase ID:

Create Purchase

Product Type: Quantity: Total Cost:

Date:

To create a Purchase:

1. In the “Create Purchase” Section click the Drop down button for Product Type to select the product to purchase
 - If you can’t see a particular product, press the “Refresh” button and try again

2. Once you have selected the Product Type, click the drop down button for Quantity and select the quantity to purchase
3. Then the next two fields will auto populate with the price and today's date
4. Then click "Create Purchase" to create the Purchase and it will appear in the table at the top

To cancel a Purchase:

1.
In the "Cancel Purchase" section, click the drop down button to select the Purchase ID of the Purchase you want to cancel
 - If you can't see your particular purchase, press the "Refresh" button and try again
2. Once you have selected the Purchase ID, click the "Cancel Purchase" button and you will see it removed from the top

1.3.3 3) Sales Breakdown Page

If you click the Sales Button on the Home Page, you'll be taken to the Sales Breakdown Page.

This page displays, by product in the Merchandise table, the total sold for each product and the amount to be paid to the touring band and the amount that Phoenix gets to keep as commission.

This page can be used by managers to track the breakdown of sales per product.

The screenshot displays the 'Sales Breakdown' interface of the Phoenix Merchandise Application. At the top, the title 'Sales Breakdown' is centered. Below it is a table with the following data:

Product Type	Total Sold	Total Cost	Total to pay Phoenix	Total to pay Band
T-Shirt	5	29.95	7.49	22.46

Below the table, there is a 'Cost' section with a label 'Total to pay Band' and a text input field containing the value '22.46'. At the bottom of the page, there are two buttons: 'Refresh Page' and 'Back to Home Page'.

IMPORTANT:

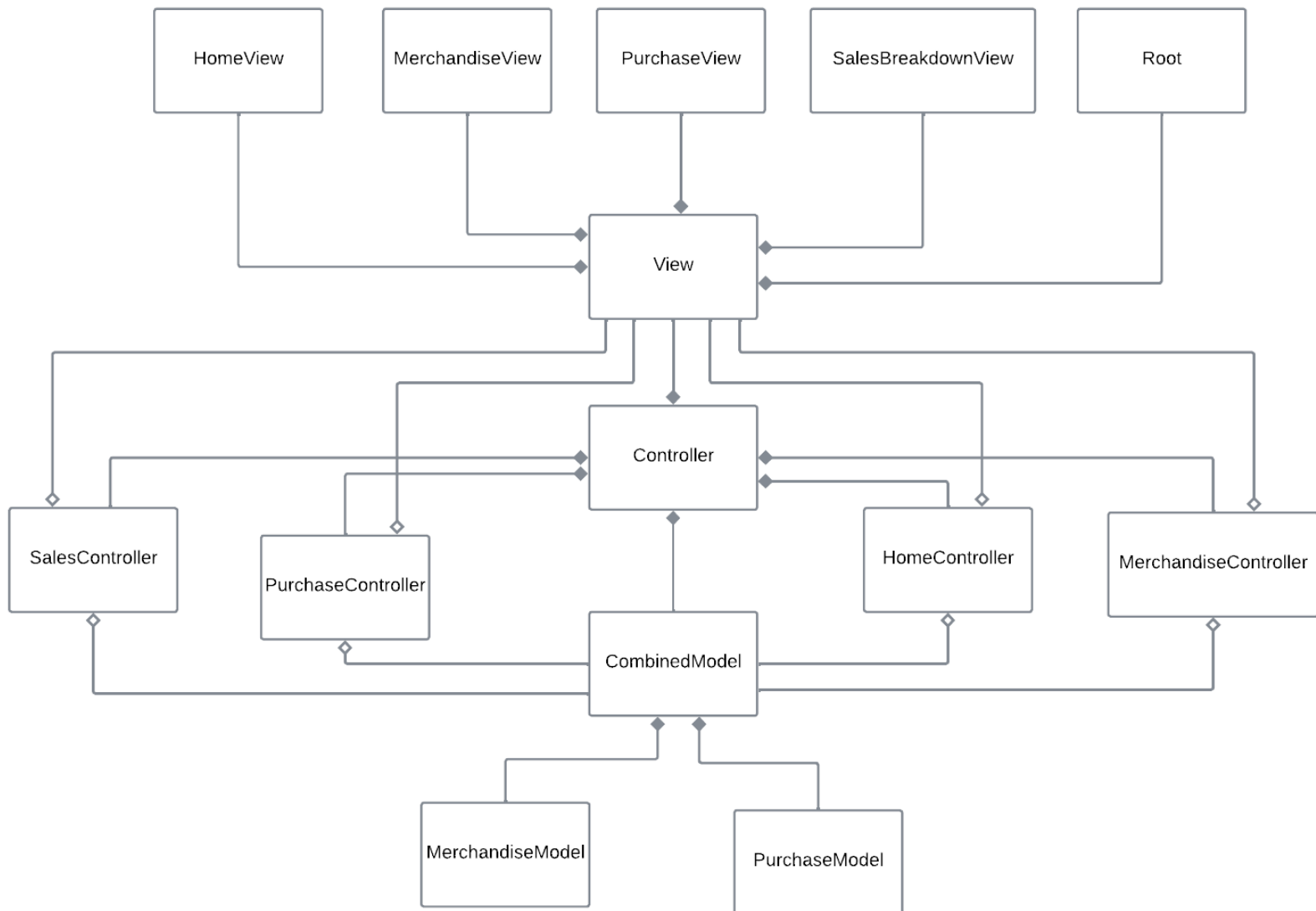
For all the pages, sometimes the application takes a while to update for new changes in data. To update the page with new data, press the "Refresh" Button. If a page is meant to contain data and it doesn't show anything, then press the "Refresh" Button and the data will appear.

1.4 Software Design

This section details the software design, so that new developers can understand how the application is structured and can then make changes or add new features to the existing application if necessary.

The application makes use of the Model-View-Controller (MVC) Design Pattern.

Below is a class diagram which shows how this design pattern has been implemented.



1.4.1 View Explanation

The purpose of the Views is to display the UI of the Phoenix application, this is where all the frontend is held. The View class is a container class for each of the specific views for each page.

The View Class holds:

- HomeView = the view for the home page
- MerchandiseView = the view for the merchandise page
- PurchaseView = the view for the purchase page
- SalesBreakdownView = the view for the sales breakdown page
- Root = this defines the root tkinter window which each one of the frames/views above will be displayed onto

Having a container class means that all the Views can be accessed from one central place which makes it easier for input to be passed from the view to the controller and from the controller to the view. The View container class controls which view to display on the root window depending on what input it receives from the controller.

1.4.2 Controller Explanation

The purpose of the Controllers is to act as the brain of the application. It will receive input, from the user who interacts with the UI, and then based on what input it receives trigger something else to execute. Usually, the controller receives input and then makes a call to a model which carries out the data processing. The Controller class acts as a container class for each of the controllers specific to each page. All controller objects are stored in this container Controller class to keep them stored centrally in an easy to access place.

The Controller Class holds:

- HomeController = the controller for the Home Page
- MerchandiseController = the controller for the Merchandise Page
- PurchaseController = the controller for Purchase Page
- SalesController = the controller for the Sales Breakdown Page

So each one of these controllers, will control the trigger of events based on input for its specific page. ie. The MerchandiseController will receive input from the MerchandiseView and then based on the input it receives make calls to the MerchandiseModel.

1.4.3 Model Explanation

The purpose of the CombinedModel Class is to act as a container for all the Model objects.

The models carry out all the data processing the application and Database CRUD operations. Each model represents a table in the Phoenix Database. Below is an entity relationship diagram which shows their relationship.



PK = Primary Key
FK = Foreign Key

There are 2 models which are stored in the CombinedModel Class:

- MerchandiseModel = This represents the Merchandise table
- PurchaseModel = This represents the Purchase table

1.5 Maintaining Software

The application uses the following software:

- Python
- tkinter
- sqlite3

- pytest (to run tests)
- flake8 (linter)

To add new features, use the Class Diagram above to help guide you, but the main steps for adding a new page are:

1. Create a <PageName>View for the new Page you would like to add, which contains all the tkinter code for how the page should look
2. Ensure that the class created above is stored in the main View class where all the View objects are held centrally
3. Create a <PageName>Model class, which carries out all the data processing for that page, and ensure it's held in the CombinedModel class along with the other models
4. Create a <PageName>Controller class which handles the input received from the <PageName>View and data after it's been processed from the <PageName>Model
5. Ensure <PageName>Controller is created and held in the main Controller class

To test new or existing features:

1. Add unit tests in the tests folder found in the directory mycode
2. To run the unit tests, use the following command after cd into mycode:

```
python -m pytest --cov=. tests/
```

3. After running the unit tests, remember to delete any temporary databases created
4. To test the frontend of the application, use the User Acceptance Testing Document found in the tests folder which documents a series of GIVEN-THEN-WHEN statements. Work through each statement and check that the application can do carry out the defined action.

1.6 References

A list of references used to create the Phoenix Merchandise Application

Links:

- <https://www.youtube.com/watch?v=YTqDYmfccQU&t=643s>
- <https://nazmul-ahsan.medium.com/how-to-organize-multi-frame-tkinter-application-with-mvc-pattern-79247ef1>
- <https://www.giacomodebidda.com/posts/mvc-pattern-in-python-sqlite/>
- <https://stackoverflow.com/questions/74787850/generating-documentation-for-multiple-folders-with-sphinx>

Application Code

2.1 main module

```
main.main ( )
```

Creates logger object, instantiates Controller, then starts Application.

2.2 constants module

Stores constants for application.

2.3 views package

2.3.1 Submodules

views.home_view module

```
class views.home_view.HomeView ( *args, **kwargs )
```

Bases: Frame

The HomeView Class creates the Home Page. It inherits from Tkinter Frame.

views.merchandise_view module

```
class views.merchandise_view.MerchandiseView ( *args, **kwargs )
```

Bases: Frame

The MerchandiseView Class creates the Merchandise Page It inherits from Tkinter Frame.

views.purchase_view module

```
class views.purchase_view.PurchaseView ( *args, **kwargs )
```

Bases: Frame

The PurchaseView class creates the Purchase Page. It inherits from Tkinter Frame.

views.root module

```
class views.root.Root
```

Bases: Tk

Creates the Tkinter root window for application.

views.sales_breakdown_view module

class views.sales_breakdown_view.**SalesBreakdownView** (**args, **kwargs*)

Bases: Frame

The SalesBreakdownView creates the Sales Breakdown Page. It inherits from Tkinter Frame.

views.view_main module

class views.view_main.**View**

Bases: object

The View class acts as a container class for all views. Pages are created and stored in this class. This class will raise a specific page(frame) to the front of the tkinter window to display it

start_main_loop ()

Method used to start the Tkinter event loop.

switch (*name: str*)

Method used to switch the page displayed on application window.

Parameters **name** (*str*) – Name of the page to display

2.3.2 Module contents

2.4 controllers package

2.4.1 Submodules

controllers.controller_main module

class controllers.controller_main.**Controller** (*db_name: str*)

Bases: object

The Controller class acts a base class for the other controllers which are specific to each page. This class stores all the controller objects.

connect_to_db (*filename: str*) → Connection

Connects to Phoenix Database and enables foreign keys

Parameter:

filename(str) : Name of file to connect to

Returns connection

Return type Sqlite3 Connection Object

start ()

Displays Home Page as first page when application starts and starts main loop.

controllers.home_controller module

class controllers.home_controller.**HomeController** (*view: View*)

Bases: object

HomeController class is the controller for the Home Page. It deals with all user input from the Home Page and makes different calls for these events.

display_merchandise_page ()

Calls the switch method from the View class to display the Merchandise page.

display_purchase_page ()

Calls the switch method from the View class to display the Purchase Page.

controllers.merchandise_controller module

class controllers.merchandise_controller.**MerchandiseController** (*view: View, model: CombinedModel*)

Bases: object

MerchandiseController is the controller for the Merchandise Page. It deals with all user input from the Merchandise page and makes different calls for these events with most calls being to the MerchandiseModel.

controllers.purchase_controller module

class controllers.purchase_controller.**PurchaseController** (*view: View, model: CombinedModel*)

Bases: object

The PurchaseController is the controller for the Purchase Page. It receives input from the Purchase Page and makes calls based on the input, usually makes calls to PurchaseModel

read_product_types ()

Reads in the Product Types from the Merchandise table into the buttons in the PurchaseView

controllers.sales_breakdown_controller module

class controllers.sales_breakdown_controller.**SalesController** (*view: View, model: CombinedModel*)

Bases: object

The SalesController class acts as a controller for the Sales Page. It receives input from the Sales-Breakdown View and then calls the corresponding event handlers.

2.4.2 Module contents**2.5 Models package****2.5.1 Submodules****CombinedModel Module**

class models.combined_model.**CombinedModel** (*connection: Connection*)

Bases: object

The CombinedModel Class acts as a container class for all the models. It instantiates the models specific to the Purchase and Merchandise Page.

MerchandiseModel Module

exception models.merchandise_model.**InvalidRecordException**

Bases: Exception

Custom Exception for when a record does not exist

class models.merchandise_model.**MerchandiseModel** (*connection: Connection*)

Bases: object

MerchandiseModel performs all data processing and CRUD Database Operations for the Merchandise Page and Merchandise Table in the Database.

add_record ()

Uses the models attributes to add a new record to the Merchandise table and then resets the attribute values so it's ready to for the next operation.

create_merchandise_table ()

Creates Merchandise table in sqlite database.

delete_merchandise_table ()

Deletes Merchandise Table when user wants to clear DB.

delete_record ()

Deletes record from Merchandise table using the id attribute. First checks whether record exists before performing delete operation.

property description: str

property id: str | int

increase_merch_quantity (id: int, quantity_to_add: int)

Method used to update quantity of a product when a purchase has been cancelled, so the quantity of the product needs to be increased again.

Parameters

- **id** (*int*) – The product id to update
- **quantity_to_add** (*int*) – The number to add to the existing product quantity

insert_initial_merchandise (data)

Inserts 3 initial records into Merchandise table so the Merchandise table already has some products when application is first started.

property price: str | float

property product_type: str

property quantity: str | int

retrieve_all_product_types () → list[tuple]

Selects and returns all product types in Merchandise table

Returns product_types

Return type list[tuple]

retrieve_all_records () → list[tuple]

Returns all the records in the Merchandise Table

Returns records

Return type List[tuple]

retrieve_count (product_type: str) → int

Returns the quantity for a specific product type

Parameter:

product_type(str) : The product type

Returns count

Return type int

retrieve_description (id: int) → str

Returns description for specific product id

Parameters `id (int)` – The product id

Returns description

Return type str

retrieve_id (*product_type: str*) → int

Returns id for specific product_type

Parameters `product_type (str)` – The product type

Returns id

Return type int

retrieve_old_quantity ()

Method used when working out the new quantity of the product after a purchase has been made or cancelled.

Sets the quantity attribute

retrieve_price (*product_type: str*) → float

Returns price for specific product type

Parameters `product_type (str)` – The product type

Returns price

Return type float

retrieve_product_type (*id: int*) → str

Returns product type for specific id

Parameters `id (int)` – The product id

Returns product_type

Return type str

retrieve_quantity (*product_type: str*) → int

Returns quantity for specific product_type

Parameters `product_type (str)` – The product type

Returns quantity

Return type int

update_quantity (*product_type: str, new_quantity: int*)

Method used to update the quantity of product after a purchase has been made or cancelled

Parameters • `product_type (str)` – The product type to update

• `new_quantity (int)` – The new quantity of the product

update_quantity_using_id ()

Updates the quantity of a product using it's product id

update_record ()

Updates record using the attribute values that correspond to Merchandise table fields. Resets the attribute values at the end so ready for next operation.

PurchaseModel Module

exception `models.purchase_model.InvalidRecordException`

Bases: `Exception`

Custom Exception to be raised for a record that does not exist.

```
class models.purchase_model.PurchaseModel ( connection: Connection )
```

Bases: object

PurchaseModel is a model for the Purchase Page. Carries out all data processing and database CRUD operations for the Purchase table.

```
calculate_cost_and_date ( price ) → tuple[str, str]
```

Method that calculates the total price of the purchase and the date purchase was made.

Parameters `price` (*float*) – The price of the product type

Returns `total_cost, date_today`

Return type tuple[str,str]

```
calculate_new_quantity ( old_quantity: int ) → int
```

Method used to calculate the new quantity for a product type after a purchase has been made

Parameters `old_quantity` (*int*) – The current quantity of the product type

Returns `new_total`

Return type int

```
calculate_sales ( ) → tuple[list[tuple], float]
```

Method used for sales breakdown page to work out the amount to be paid to the band and phoenix by product.

Returns `results, total_cost_for_band`

Return type tuple[list[tuple], float]

```
create_purchase_table ( )
```

Creates Purchase table in sqlite database.

```
create_record ( )
```

Creates record in Purchase table using attribute values that correspond to Purchase table fields. Then resets attribute values so ready for next operation.

property date: str

```
delete_purchase_table ( )
```

Deletes Purchase table when clear DB.

```
delete_record ( )
```

Deletes record from Purchase table using id attribute. First checks if record to delete actually exists.

property id: int | str

property merchandise_id: int | str

property quantity: int | str

```
retrieve_all_ids ( ) → list[tuple]
```

Returns all IDs in Purchase table

Returns `ids`

Return type list[tuple]

```
retrieve_all_records ( ) → list[tuple]
```

Returns all records in Purchase table

Returns `records`

Return type list[tuple]

retrieve_merch_id() → int

Returns merch_id for a purchase

Returns merchandise_id

Return type int

retrieve_quantity() → int

Returns the quantity in a particular purchase

Returns quantity

Return type int

property total_cost: float | str

2.5.2 Module contents

- genindex
- modindex
- search

C

constants, [11](#)
controllers, [13](#)
 controllers.controller_main, [12](#)
 controllers.home_controller, [12](#)
 controllers.merchandise_controller,
 [13](#)
 controllers.purchase_controller,
 [13](#)
 controllers.sales_breakdown_controller,
 [13](#)

m

main, [11](#)
models, [17](#)
 models.combined_model, [13](#)
 models.merchandise_model, [13](#)
 models.purchase_model, [15](#)

V

views, [12](#)
 views.home_view, [11](#)
 views.merchandise_view, [11](#)
 views.purchase_view, [11](#)
 views.root, [11](#)
 views.sales_breakdown_view, [12](#)
 views.view_main, [12](#)

A

`add_record()` (models.merchandise_model.MerchandiseModel method), 14

C

`calculate_cost_and_date()` (models.purchase_model.PurchaseModel method), 16

`calculate_new_quantity()` (models.purchase_model.PurchaseModel method), 16

`calculate_sales()` (models.purchase_model.PurchaseModel method), 16

`CombinedModel` (class in models.combined_model), 13

`connect_to_db()` (controllers.controller_main.Controller method), 12

`constants`
module, 11

`Controller` (class in controllers.controller_main), 12

`controllers`
module, 13

`controllers.controller_main`
module, 12

`controllers.home_controller`
module, 12

`controllers.merchandise_controller`
module, 13

`controllers.purchase_controller`
module, 13

`controllers.sales_breakdown_controller`
module, 13

`create_merchandise_table()` (models.merchandise_model.MerchandiseModel method), 14

`create_purchase_table()` (models.purchase_model.PurchaseModel method), 16

`create_record()` (models.purchase_model.PurchaseModel method), 16

D

`date` (models.purchase_model.PurchaseModel

property), 16

`delete_merchandise_table()` (models.merchandise_model.MerchandiseModel method), 14

`delete_purchase_table()` (models.purchase_model.PurchaseModel method), 16

`delete_record()` (models.merchandise_model.MerchandiseModel method), 14

`delete_record()` (models.purchase_model.PurchaseModel method), 16

`description` (models.merchandise_model.MerchandiseModel property), 14

`display_merchandise_page()` (controllers.home_controller.HomeController method), 12

`display_purchase_page()` (controllers.home_controller.HomeController method), 13

H

`HomeController` (class in controllers.home_controller), 12

`HomeView` (class in views.home_view), 11

I

`id` (models.merchandise_model.MerchandiseModel property), 14

`id` (models.purchase_model.PurchaseModel property), 16

`increase_merch_quantity()` (models.merchandise_model.MerchandiseModel method), 14

`insert_initial_merchandise()` (models.merchandise_model.MerchandiseModel method), 14

`InvalidRecordException`, 13, 15

M

`main`
module, 11

`main()` (in module main), 11

`merchandise_id` (models.purchase_model.Pur-

- chaseModelProperty), 16
- MerchandiseController (class in controllers.merchandise_controller), 13
- MerchandiseModel (class in models.merchandise_model), 13
- MerchandiseView (class in views.merchandise_view), 11
- models
 - module, 17
- models.combined_model
 - module, 13
- models.merchandise_model
 - module, 13
- models.purchase_model
 - module, 15
- module
 - constants, 11
 - controllers, 13
 - controllers.controller_main, 12
 - controllers.home_controller, 12
 - controllers.merchandise_controller, 13
 - controllers.purchase_controller, 13
 - controllers.sales_breakdown_controller, 13
 - main, 11
 - models, 17
 - models.combined_model, 13
 - models.merchandise_model, 13
 - models.purchase_model, 15
 - views, 12
 - views.home_view, 11
 - views.merchandise_view, 11
 - views.purchase_view, 11
 - views.root, 11
 - views.sales_breakdown_view, 12
 - views.view_main, 12

P

- price (models.merchandise_model.MerchandiseModel property), 14
- product_type (models.merchandise_model.MerchandiseModel property), 14
- PurchaseController (class in controllers.purchase_controller), 13
- PurchaseModel (class in models.purchase_model), 16
- PurchaseView (class in views.purchase_view), 11

Q

- quantity (models.merchandise_model.MerchandiseModel property), 14
- quantity (models.purchase_model.PurchaseModel property), 16

R

- read_product_types() (controllers.purchase_controller.PurchaseController

- method), 13
- retrieve_all_ids() (models.purchase_model.PurchaseModel method), 16
- retrieve_all_product_types() (models.merchandise_model.MerchandiseModel method), 14
- retrieve_all_records() (models.merchandise_model.MerchandiseModel method), 14
- retrieve_all_records() (models.purchase_model.PurchaseModel method), 16
- retrieve_count() (models.merchandise_model.MerchandiseModel method), 14
- retrieve_description() (models.merchandise_model.MerchandiseModel method), 14
- retrieve_id() (models.merchandise_model.MerchandiseModel method), 15
- retrieve_merch_id() (models.purchase_model.PurchaseModel method), 17
- retrieve_old_quantity() (models.merchandise_model.MerchandiseModel method), 15
- retrieve_price() (models.merchandise_model.MerchandiseModel method), 15
- retrieve_product_type() (models.merchandise_model.MerchandiseModel method), 15
- retrieve_quantity() (models.merchandise_model.MerchandiseModel method), 15
- retrieve_quantity() (models.purchase_model.PurchaseModel method), 17
- Root (class in views.root), 11

S

- SalesBreakdownView (class in views.sales_breakdown_view), 12
- SalesController (class in controllers.sales_breakdown_controller), 13
- start() (controllers.controller_main.Controller method), 12
- start_main_loop() (views.view_main.View method), 12
- switch() (views.view_main.View method), 12

T

- total_cost (models.purchase_model.PurchaseModel property), 17

U

- update_quantity() (models.merchandise_model.MerchandiseModel method), 15
- update_quantity_using_id() (models.merchandise_model.MerchandiseModel method), 15
- update_record() (models.merchandise_model.MerchandiseModel method), 15

V

View (class in views.view_main), [12](#)

views

 module, [12](#)

views.home_view

 module, [11](#)

views.merchandise_view

 module, [11](#)

views.purchase_view

 module, [11](#)

views.root

 module, [11](#)

views.sales_breakdown_view

 module, [12](#)

views.view_main

 module, [12](#)

